

Simple Programs



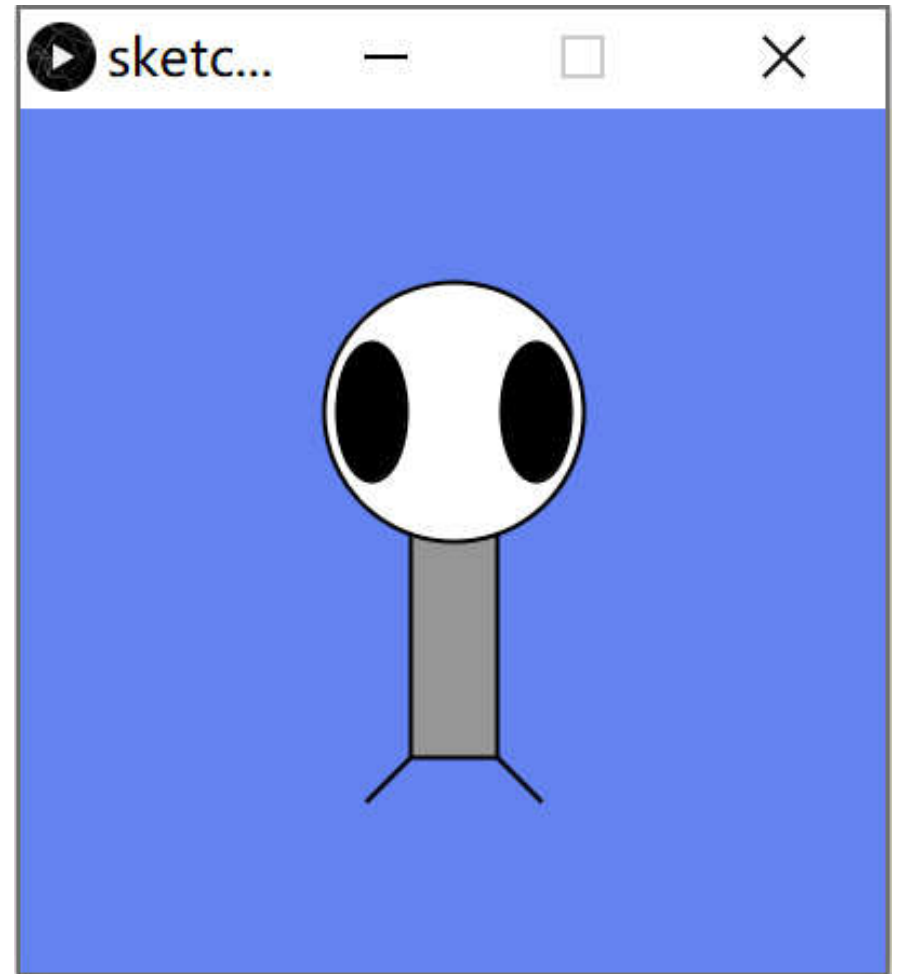
YJ40201: Fundamentals of Programming
Stony Brook Institute at Anhui University

Today's Agenda

- **Writing a simple program in Processing**
- **Introduction to algorithms**
- **Introduction to functions**
 - **setup() and draw()**
- **Relevant reading: Chapters 2 and 3**

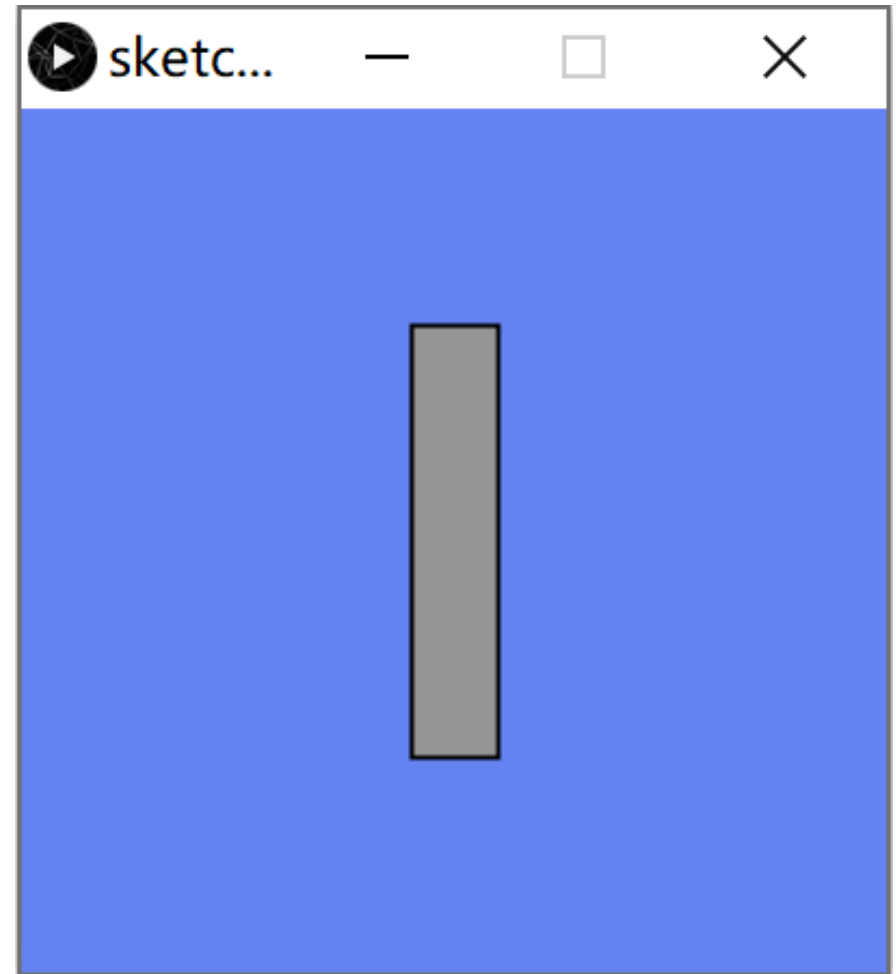
Meet Zoog

- Zoog is a small creature designed with Processing
- Zoog is drawn using a rectangle, three ellipses, and two lines
- We'll use Zoog to try out new programming concepts



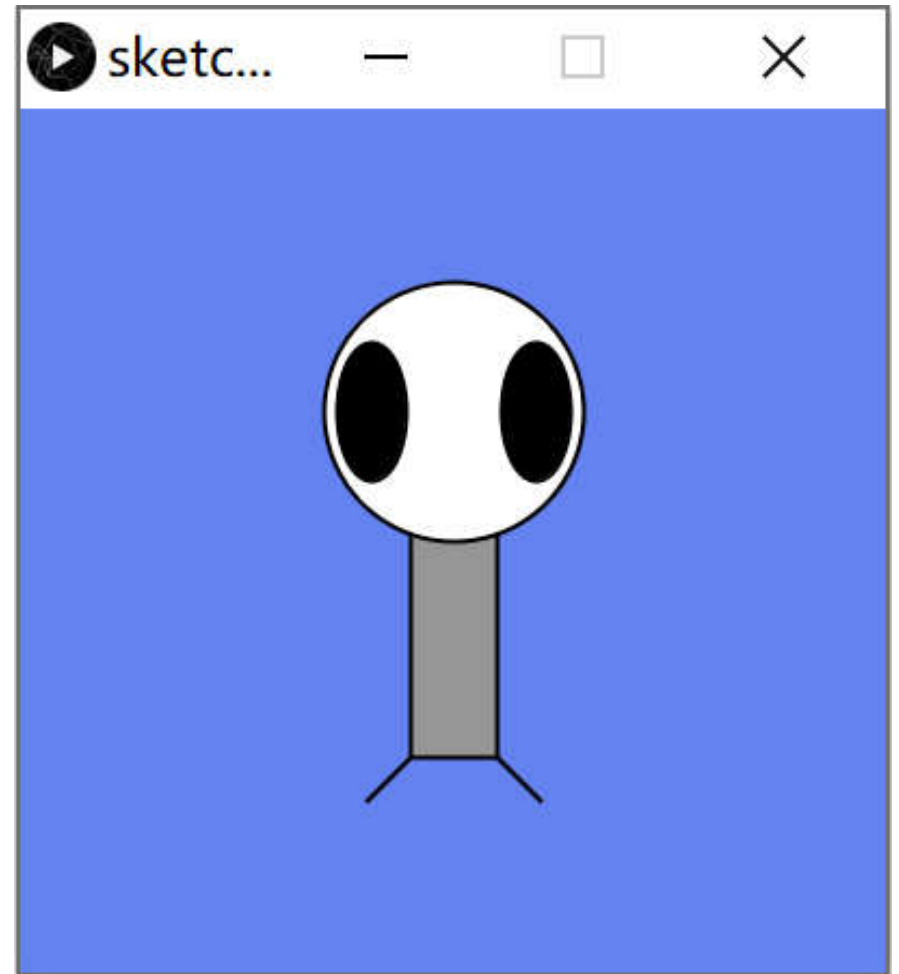
Drawing Zoog, Part 1

```
size(200, 200);  
background(100, 130, 240);  
  
ellipseMode(CENTER);  
rectMode(CENTER);  
// Get rid of jaggy lines  
smooth();  
// Draw Zoog's body  
stroke(0);  
fill(150);  
rect(100,100,20,100);
```



Drawing Zoog, Part 2

```
// Draw Zoog's head  
fill(255);  
ellipse(100,70,60,60);  
  
// Draw Zoog's eyes  
fill(0);  
ellipse(81,70,16,32);  
ellipse(119,70,16,32);  
  
// Draw Zoog's legs  
stroke(0);  
line(90,150,80,160);  
line(110,150,120,160);
```



Algorithms



Algorithms

- **An algorithm is a description of the steps necessary to solve a problem**
 - **“problem” = “task to be performed”**
- **Algorithms are the heart of computer science**
- **A computer program embodies (carries out) an algorithm**

Algorithm Examples

- **Grandma's recipe for chocolate chip cookies**
- **Driving directions**
- **Putting together a class schedule**
- **Playing a board game (e.g., Monopoly)**

Algorithm Attributes

- An algorithm must be *complete*
 - It must describe *every* step of the solution
- An algorithm must be *precise*
 - It should tell you *exactly* what to do for a given step

Developing an Algorithm

- **Start with the input and desired output**
- **Divide and conquer the problem**
 - **Break it up into smaller pieces**
- **Iterative refinement of solution**
 - **Keep applying divide and conquer to each sub-step**

Algorithm Building Blocks

- **1. Expressions**
- **2. Conditional (selection) statements**
- **3. Iteration (repetition) statements**
- **4. Subprograms (methods)**
 - **These are previously-defined algorithms**

An Algorithm Example

- **What's a good algorithm for making a peanut butter and jelly sandwich?**
 - **HINT: start with the ingredients (input)**
- **A good rule of thumb for algorithms is to imagine that you need to tell a 3-year-old how to perform the task**

Another Example

- **Problem: Given two integers, find their greatest common divisor (the largest number that divides evenly into both)**
- **The best-known algorithm for this was developed by Euclid**

Euclid's GCD Algorithm

- 1. Let A be the larger of the two integers**
- 2. Let B be the smaller of the two integers**
- 3. If B is 0, return A then stop.**
- 4. Otherwise, replace A with B , replace B with the remainder of (A divided by B)**
- 5. Go to Step 3**

Functions



What's a Function?

- **A *function* (also called a *method*) is a block (group) of program instructions**
 - **A name is assigned to this block**
 - **The block is surrounded by curly braces**
- **Whenever we call the function's name, that set of instructions is executed**
 - **This lets us do something many times, but only write the code one time**

Functions We've Seen

- **line()**
- **rect()**
- **ellipse()**
- **size()**
- **background()**
- **etc.**

Function Input and Output

- A function can take one or more values as *arguments* (input)
 - e.g., `line()` takes two sets of coordinates
 - Arguments are placed in the parentheses after the function's name
- A function can return a value as its output
 - A *void* function does not return a value

setup() and draw()

- Most Processing programs have two special functions: **setup()** and **draw()**
 - These functions are useful for animation
- **setup()** is called *once*, when the program starts
- **draw()** is called *over and over*, and puts new information on the screen
 - e.g., the next frame of animation

Flow of Control

```
void setup ()  
{  
    // Step 1a  
    // Step 1b  
    // Step 1c  
}
```

```
void draw ()  
{  
    // Step 2a  
    // Step 2b  
}
```

**These instructions will
be executed as follows:**

**1a, 1b, 1c, 2a, 2b, 2a, 2b,
2a, 2b, 2a, 2b, 2a, 2b, ...**

Drawing Zoog With Functions

- Remember that `setup()` only runs once
 - Use this function for things that don't change

```
void setup ()  
{  
    size(210, 200);  
    background(80, 100, 240);  
  
    ellipseMode(CENTER);  
    rectMode(CENTER);  
    smooth(); // Get rid of jaggy lines  
}
```

Another Zoog Function

- The draw() function is called repeatedly
 - Use it for things that (may) change
- Right now, nothing seems to change
 - We'll fix that soon...

```
void draw ()
{
    // Draw Zoog's body
    stroke(0);
    fill(150);
    rect(100,100,20,100);
    // Draw Zoog's head
    fill(255);
    ellipse(100,70,60,60);
    // Draw Zoog's eyes
    fill(0);
    ellipse(81,70,16,32);
    ellipse(119,70,16,32);
    // Draw Zoog's legs
    stroke(0);
    line(90,150,80,160);
    line(110,150,120,160);
}
```

Next Time

- **Working with the mouse**
 - **Mouse positions**
 - **Mouse clicks**
- **Variables**
 - **How to store and work with information**